

Facultad: Ciencias de la Ingeniería

Asignatura: Fundamentos de la Especialidad II

Docente: Mg. Viviana Flores

Estudiante: Raul Fernando Cajiao Garces

Actividad Práctica: Auditoría y Refactorización de Código

Aplicación de Principios de Diseño SOLID, DRY, KISS e ISO/IEC 25010

1 Introducción y Objetivos

El desarrollo de software moderno no solo requiere que el código sea funcional, sino que también cumpla con estándares de calidad que faciliten su mantenimiento, escalabilidad y testabilidad a lo largo del tiempo.

Esta actividad práctica tiene como objetivo auditar un componente denominado *GestorPedidos*, identificar violaciones de diseño clave a través del análisis de principios SOLID, DRY y KISS, y aplicar un proceso de refactorización estructural. Asimismo, se evalúan las mejoras del código utilizando las subcaracterísticas de la norma internacional **ISO/IEC 25010** (específicamente modularidad, reusabilidad, analizabilidad, modificabilidad y testabilidad), contrastando el estado original con la arquitectura propuesta.

2 Enlaces del Entregable

A continuación se detallan los enlaces correspondientes al desarrollo del laboratorio:

- **Repositorio de GitHub:** https://github.com/Raul-Fernando-Cajiao-Garces/DEBER1_FUNDAMETOS2
- **Video Demostrativo (Explicación técnica):** <https://youtu.be/video-demostrativo-placeholder>

3 Código Original Auditado

El siguiente listado muestra el código original en Java de la clase GestorPedidos, la cual contiene múltiples violaciones a buenas prácticas y principios de diseño de software.

```
1 import java.util.*;
2 import java.io.*;
3 import java.sql.*;
4
5 public class GestorPedidos {
6     private Connection conexionBD;
7
8     public GestorPedidos() {
9         try {
10             this.conexionBD = DriverManager.getConnection(
11                 "jdbc:mysql://localhost:3306/tienda", "root", "admin123
12             ");
13         } catch (SQLException e) {
14             e.printStackTrace();
15         }
16     }
17
18     public void procesarPedido(String nombreCliente, String
19     emailCliente,
20
21     List<String> nombresProductos,
22     List<Double> preciosProductos,
23     List<Integer> cantidades,
24     String tipoCliente) {
25         if (nombreCliente == null || nombreCliente.trim().isEmpty()) {
26             System.out.println("Error: nombre de cliente invalido");
27             return;
28         }
29         if (emailCliente == null || !emailCliente.contains("@")) {
30             System.out.println("Error: email invalido");
31             return;
32         }
33
34         double subtotal = 0;
35         for (int i = 0; i < nombresProductos.size(); i++) {
```

```

33         subtotal += preciosProductos.get(i) * cantidades.get(i);
34     }
35
36     double descuento = 0;
37     if (tipoCliente.equals("VIP")) {
38         descuento = subtotal * 0.20;
39     } else if (tipoCliente.equals("FRECUENTE")) {
40         descuento = subtotal * 0.10;
41     } else if (tipoCliente.equals("REGULAR")) {
42         descuento = subtotal * 0.05;
43     } else if (tipoCliente.equals("NUEVO")) {
44         descuento = 0;
45     }
46
47     double impuesto = (subtotal - descuento) * 0.12;
48     double total = subtotal - descuento + impuesto;
49
50     try {
51         Statement stmt = conexionBD.createStatement();
52         String sql = "INSERT INTO pedidos (cliente, total) VALUES
53 ('"
54         + nombreCliente + "', " + total + ")";
55         stmt.executeUpdate(sql);
56     } catch (SQLException e) {
57         System.out.println("Error al guardar el pedido: " + e.
58 getMessage());
59     }
60
61     try {
62         FileWriter writer = new FileWriter("factura_" +
63 nombreCliente + ".txt");
64         writer.write("FACTURA\n");
65         writer.write("Cliente: " + nombreCliente + "\n");
66         for (int i = 0; i < nombresProductos.size(); i++) {
67             writer.write(nombresProductos.get(i) + " x" +
68 cantidades.get(i)
69             + " = $" + (preciosProductos.get(i) * cantidades.
70 get(i)) + "\n");
71         }

```

```

67         writer.write("Subtotal: $" + subtotal + "\n");
68         writer.write("Descuento: $" + descuento + "\n");
69         writer.write("Impuesto: $" + impuesto + "\n");
70         writer.write("TOTAL: $" + total + "\n");
71         writer.close();
72     } catch (IOException e) {
73         System.out.println("Error al generar la factura: " + e.
getMessage());
74     }
75
76     System.out.println("Enviando correo a " + emailCliente + "...")
;
77     System.out.println("Asunto: Confirmacion de pedido");
78     System.out.println("Cuerpo: Estimado " + nombreCliente + ", su
pedido por $"
79         + total + " ha sido procesado.");
80     System.out.println("[LOG] Pedido procesado para " +
nombreCliente
81         + " - Total: " + total);
82 }
83
84 public void cancelarPedido(String nombreCliente, String
emailCliente, int idPedido) {
85     if (nombreCliente == null || nombreCliente.trim().isEmpty()) {
86         System.out.println("Error: nombre de cliente invalido");
87         return;
88     }
89     if (emailCliente == null || !emailCliente.contains("@")) {
90         System.out.println("Error: email invalido");
91         return;
92     }
93
94     try {
95         Statement stmt = conexionBD.createStatement();
96         String sql = "DELETE FROM pedidos WHERE id = " + idPedido;
97         stmt.executeUpdate(sql);
98     } catch (SQLException e) {
99         System.out.println("Error al cancelar el pedido: " + e.
getMessage());

```

```
100     }
101
102     System.out.println("Enviando correo a " + emailCliente + "...")
103     ;
104     System.out.println("Asunto: Cancelacion de pedido");
105     System.out.println("Cuerpo: Estimado " + nombreCliente + ", su
106     pedido #"
107         + idPedido + " ha sido cancelado.");
108 }
```

Listing 1: Código Java Original

4 Identificación de Violaciones de Principios (Auditoría)

A continuación, se detalla la auditoría del código original mediante la identificación de violaciones específicas a los principios SOLID, DRY y KISS:

Cuadro 1: Tabla de Violaciones de Principios de Diseño

Principio	Líneas / Ubicación	Descripción Técnica de la Violación
SRP (Single Responsibility Principle)	Clase completa	La clase tiene múltiples responsabilidades: conectarse a la base de datos, validar parámetros, calcular precios, registrar transacciones en SQL, generar facturas en texto plano, notificar a los clientes y registrar logs en consola. Cualquier cambio en estas áreas requiere modificar la misma clase.
OCP (Open/Closed Principle)	Líneas 39 a 48	La lógica de descuentos depende de un árbol de condicionales if-else basado en el parámetro tipoCliente. Si se añade un nuevo tipo de cliente (e.g., PREMIUM), se debe alterar la estructura lógica de la clase, exponiendo el código a errores de regresión.
DIP (Dependency Inversion Principle)	Constructor y Métodos	Hay un alto acoplamiento a implementaciones concretas de infraestructura: se invoca directamente <code>DriverManager.getConnection</code> , <code>FileWriter</code> y las operaciones SQL vía <code>Statement</code> con concatenación de strings, impidiendo mockear o sustituir la base de datos o el sistema de archivos durante pruebas unitarias.
DRY (Don't Repeat Yourself)	Líneas 19-26 y 87-94 / Líneas 79-84 y 99-103	Las validaciones de cliente (<code>nombreCliente</code> y <code>emailCliente</code>) se repiten textualmente tanto en <code>procesarPedido</code> como en <code>cancelarPedido</code> . Adicionalmente, el formato de envío de correo se repite en ambos flujos.

Cuadro 1: Tabla de Violaciones de Principios de Diseño
(continuación)

Principio	Líneas / Ubicación	Descripción Técnica de la Violación
KISS (Keep It Simple, Stupid)	Métodos de negocio	Los métodos <code>procesarPedido</code> y <code>cancelarPedido</code> mezclan lógica de validación, computación matemática, persistencia de red e I/O de disco. Esto vuelve compleja la lectura del código, y propicia posibles fallos difíciles de depurar en un único bloque monolítico.
Seguridad (Extra)	Líneas 50-52 y 90-91	Vulnerabilidad de Inyección SQL debido a la concatenación directa de parámetros en sentencias ejecutadas por <code>Statement</code> en lugar de usar <code>PreparedStatement</code> . Además, las credenciales de la base de datos están quemadas (<i>hardcoded</i>).

5 Código Refactorizado Propuesto

Para solventar las deficiencias identificadas, el código fue separado en módulos según su responsabilidad, inyectando dependencias por medio de abstracciones (interfaces) y aplicando el patrón de diseño **Strategy** para los descuentos.

5.1 1. Abstracciones e Interfaces

```
1 package refactorizado.interfaces;
2 import refactorizado.model.Pedido;
3 import refactorizado.model.DetallePrecios;
4
5 public interface IPedidoRepository {
6     void guardarPedido(String cliente, double total);
7     void eliminarPedido(int idPedido);
8 }
9
10 public interface IFacturador {
11     void generarFactura(Pedido pedido, DetallePrecios precios);
12 }
13
14 public interface INotificador {
15     void enviarConfirmacion(String nombreCliente, String emailCliente,
16         double total);
17     void enviarCancelacion(String nombreCliente, String emailCliente,
18         int idPedido);
19 }
20
21 public interface IDescuentoStrategy {
22     double calcularDescuento(double subtotal);
23 }
```

Listing 2: Abstracciones del Sistema

5.2 2. Patrón de Diseño Strategy (Descuentos)

```
1 package refactorizado.strategy;
```

```

2 import refactorizado.interfaces.IDescuentoStrategy;
3 import java.util.HashMap;
4 import java.util.Map;
5
6 public class DescuentoVIP implements IDescuentoStrategy {
7     public double calcularDescuento(double subtotal) { return subtotal
8         * 0.20; }
9 }
10
11 public class DescuentoFrecuente implements IDescuentoStrategy {
12     public double calcularDescuento(double subtotal) { return subtotal
13         * 0.10; }
14 }
15
16 public class DescuentoRegular implements IDescuentoStrategy {
17     public double calcularDescuento(double subtotal) { return subtotal
18         * 0.05; }
19 }
20
21 public class DescuentoNuevo implements IDescuentoStrategy {
22     public double calcularDescuento(double subtotal) { return 0.0; }
23 }
24
25 // Factoria de Estrategias para evitar condicionales
26 public class DescuentoFactory {
27     private static final Map<String, IDescuentoStrategy> estrategias =
28         new HashMap<>();
29
30     static {
31         estrategias.put("VIP", new DescuentoVIP());
32         estrategias.put("FRECUENTE", new DescuentoFrecuente());
33         estrategias.put("REGULAR", new DescuentoRegular());
34         estrategias.put("NUEVO", new DescuentoNuevo());
35     }
36
37     public static IDescuentoStrategy obtenerEstrategia(String
38         tipoCliente) {
39         return estrategias.getDefault(tipoCliente, new DescuentoNuevo
40             ());

```

35

}

36

}

Listing 3: Estrategias de Descuento (SOLID OCP)

5.3 3. Validadores y Lógica de Dominio

```
1 package refactorizado.model;
2 import java.util.List;
3
4 public class ValidadorCliente {
5     public static void validar(String nombreCliente, String
6         emailCliente) {
7         if (nombreCliente == null || nombreCliente.trim().isEmpty()) {
8             throw new IllegalArgumentException("Error: nombre de
9             cliente invalido");
10        }
11        if (emailCliente == null || !emailCliente.contains("@")) {
12            throw new IllegalArgumentException("Error: email invalido")
13        };
14    }
15 }
16
17 public class Pedido {
18     private final String nombreCliente;
19     private final String emailCliente;
20     private final List<String> nombresProductos;
21     private final List<Double> preciosProductos;
22     private final List<Integer> cantidades;
23     private final String tipoCliente;
24
25     public Pedido(String nombreCliente, String emailCliente, List<
26         String> nombresProductos,
27         List<Double> preciosProductos, List<Integer>
28         cantidades, String tipoCliente) {
29         ValidadorCliente.validar(nombreCliente, emailCliente);
30         this.nombreCliente = nombreCliente;
31         this.emailCliente = emailCliente;
32         this.nombresProductos = nombresProductos;
33         this.preciosProductos = preciosProductos;
34         this.cantidades = cantidades;
35         this.tipoCliente = tipoCliente;
36     }
37 }
```

```

33
34     public String getNombreCliente() { return nombreCliente; }
35     public String getEmailCliente() { return emailCliente; }
36     public List<String> getNombresProductos() { return nombresProductos
; }
37     public List<Double> getPreciosProductos() { return preciosProductos
; }
38     public List<Integer> getCantidades() { return cantidades; }
39     public String getTipoCliente() { return tipoCliente; }
40
41     public double calcularSubtotal() {
42         double subtotal = 0;
43         for (int i = 0; i < nombresProductos.size(); i++) {
44             subtotal += preciosProductos.get(i) * cantidades.get(i);
45         }
46         return subtotal;
47     }
48 }
49
50 public class DetallePrecios {
51     public final double subtotal;
52     public final double descuento;
53     public final double impuesto;
54     public final double total;
55
56     public DetallePrecios(double subtotal, double descuento, double
impuesto, double total) {
57         this.subtotal = subtotal;
58         this.descuento = descuento;
59         this.impuesto = impuesto;
60         this.total = total;
61     }
62 }
63
64 public class CalculadoraPrecios {
65     private static final double IVA_RATE = 0.12;
66
67     public static DetallePrecios calcular(Pedido pedido) {
68         double subtotal = pedido.calcularSubtotal();

```

```

69         double descuento = DescuentoFactory.obtenerEstrategia(pedido.
getTipoCliente())
70                                     .calcularDescuento(subtotal)
71     ;
72     double impuesto = (subtotal - descuento) * IVA_RATE;
73     double total = subtotal - descuento + impuesto;
74     return new DetallePrecios(subtotal, descuento, impuesto, total)
75 ;
76 }
77 }

```

Listing 4: Clases de Dominio, Validación y Cálculo de Precios

5.4 4. Implementaciones de Infraestructura

```
1 package refactorizado.infraestructura;
2 import refactorizado.interfaces.*;
3 import refactorizado.model.*;
4 import java.io.FileWriter;
5 import java.io.IOException;
6 import java.sql.Connection;
7 import java.sql.PreparedStatement;
8 import java.sql.SQLException;
9
10 public class PedidoRepositorySQL implements IPedidoRepository {
11     private final Connection conexionBD;
12
13     public PedidoRepositorySQL(Connection conexionBD) {
14         this.conexionBD = conexionBD;
15     }
16
17     @Override
18     public void guardarPedido(String cliente, double total) {
19         String sql = "INSERT INTO pedidos (cliente, total) VALUES (?,
20         ?)";
21         try (PreparedStatement stmt = conexionBD.prepareStatement(sql))
22         {
23             stmt.setString(1, cliente);
24             stmt.setDouble(2, total);
25             stmt.executeUpdate();
26         } catch (SQLException e) {
27             System.out.println("Error al guardar el pedido: " + e.
28             getMessage());
29         }
30     }
31
32     @Override
33     public void eliminarPedido(int idPedido) {
34         String sql = "DELETE FROM pedidos WHERE id = ?";
35         try (PreparedStatement stmt = conexionBD.prepareStatement(sql))
36         {
37             stmt.setInt(1, idPedido);
```

```

34         stmt.executeUpdate();
35     } catch (SQLException e) {
36         System.out.println("Error al cancelar el pedido: " + e.
getMessage());
37     }
38 }
39 }
40
41 public class FacturadorTexto implements IFacturador {
42     @Override
43     public void generarFactura(Pedido pedido, DetallePrecios precios) {
44         try (FileWriter writer = new FileWriter("factura_" + pedido.
getNombreCliente() + ".txt")) {
45             writer.write("FACTURA\n");
46             writer.write("Cliente: " + pedido.getNombreCliente() + "\n"
);
47             for (int i = 0; i < pedido.getNombresProductos().size(); i
++) {
48                 writer.write(pedido.getNombresProductos().get(i) + " x"
+ pedido.getCantidades().get(i)
49                     + " = $" + (pedido.getPreciosProductos().get(i) *
pedido.getCantidades().get(i)) + "\n");
50             }
51             writer.write("Subtotal: $" + precios.subtotal + "\n");
52             writer.write("Descuento: $" + precios.descuento + "\n");
53             writer.write("Impuesto: $" + precios.impuesto + "\n");
54             writer.write("TOTAL: $" + precios.total + "\n");
55         } catch (IOException e) {
56             System.out.println("Error al generar la factura: " + e.
getMessage());
57         }
58     }
59 }
60
61 public class NotificadorConsole implements INotificador {
62     @Override
63     public void enviarConfirmacion(String nombreCliente, String
emailCliente, double total) {
64         System.out.println("Enviando correo a " + emailCliente + "...")

```



```

;
65     System.out.println("Asunto: Confirmacion de pedido");
66     System.out.println("Cuerpo: Estimado " + nombreCliente + ", su
pedido por $"
67         + total + " ha sido procesado.");
68     System.out.println("[LOG] Pedido procesado para " +
nombreCliente + " - Total: " + total);
69 }
70
71 @Override
72 public void enviarCancelacion(String nombreCliente, String
emailCliente, int idPedido) {
73     System.out.println("Enviando correo a " + emailCliente + "...")
;
74     System.out.println("Asunto: Cancelacion de pedido");
75     System.out.println("Cuerpo: Estimado " + nombreCliente + ", su
pedido #"
76         + idPedido + " ha sido cancelado.");
77 }
78 }

```

Listing 5: Implementaciones Concretas

5.5 5. Orquestador Refactorizado (Clase Principal)

```
1 package refactorizado;
2 import refactorizado.interfaces.*;
3 import refactorizado.model.*;
4
5 public class GestorPedidos {
6     private final IPedidoRepository repository;
7     private final IFacturador facturador;
8     private final INotificador notificador;
9
10    public GestorPedidos(IPedidoRepository repository, IFacturador
facturador, INotificador notificador) {
11        this.repository = repository;
12        this.facturador = facturador;
13        this.notificador = notificador;
14    }
15
16    public void procesarPedido(Pedido pedido) {
17        DetallePrecios precios = CalculadoraPrecios.calcular(pedido);
18        repository.guardarPedido(pedido.getNombreCliente(), precios.
total);
19        facturador.generarFactura(pedido, precios);
20        notificador.enviarConfirmacion(pedido.getNombreCliente(),
pedido.getEmailCliente(), precios.total);
21    }
22
23    public void cancelarPedido(String nombreCliente, String
emailCliente, int idPedido) {
24        ValidadorCliente.validar(nombreCliente, emailCliente);
25        repository.eliminarPedido(idPedido);
26        notificador.enviarCancelacion(nombreCliente, emailCliente,
idPedido);
27    }
28 }
```

Listing 6: Clase GestorPedidos Refactorizada

6 Justificación de la Refactorización (ISO/IEC 25010)

El modelo de calidad de software ISO/IEC 25010 nos permite medir de forma estructurada los beneficios del proceso de refactorización sobre las características internas de mantenibilidad y modularidad.

Cuadro 2: Comparación de Características de Calidad (Antes vs. Después)

Subcaracterística	Estado Anterior (Antes)	Estado Refactorizado (Después)
Modularidad	El código original residía en una única clase monolítica altamente acoplada. Modificar la persistencia afectaba a la lógica de negocio y viceversa.	Se divide en componentes independientes con interfaces claras: persistencia, facturación, y notificaciones. Los módulos pueden ser reemplazados sin impactar al resto.
Reusabilidad	Lógica como el cálculo de descuentos o la generación de facturas estaba embebida dentro de un flujo lineal, imposible de reutilizar en otros servicios.	La calculadora de precios y las estrategias de descuento son reutilizables por cualquier otro orquestador o módulo del sistema.
Analizabilidad	Comprender el código requería rastrear conexiones SQL manuales y concatenaciones junto a fórmulas aritméticas en más de 120 líneas de código secuencial.	El código es semánticamente autoexplicativo. Cada clase tiene menos de 40 líneas de código enfocadas en una única tarea estructurada.
Modificabilidad	Agregar un tipo de descuento implicaba modificar la clase core (GestorPedidos), rompiendo potencialmente la persistencia o notificaciones.	Cumple con OCP. Para añadir un nuevo tipo de cliente solo se crea una clase que implemente IDescuentoStrategy y se registra en la fábrica, sin tocar el core.

Cuadro 2: Comparación de Características de Calidad (continuación)

Subcaracterística	Estado Anterior (Antes)	Estado Refactorizado (Después)
Testabilidad	Era imposible testear la lógica de negocio sin conectarse a una base de datos MySQL real y escribir archivos físicos en el sistema operativo.	Se permite la inyección de mocks para todas las interfaces auxiliares (IPedidoRepository, INotificador, etc.), posibilitando tests unitarios aislados del entorno externo.

7 Historial del Proceso de Refactorización (Commits)

Para demostrar la evolución e incrementalidad del proceso, se describe la secuencia planificada de confirmaciones (commits) realizadas en el repositorio de control de versiones Git:

1. **Commit 1 - [Feat]:** Añadir código original de `GestorPedidos.java` como versión inicial sin modificaciones para establecer la línea base.
2. **Commit 2 - [Refactor]:** Crear modelo de dominio `Pedido`, clase de utilidad `ValidadorCliente` y transferir la lógica de validación para eliminar código duplicado (DRY).
3. **Commit 3 - [Pattern]:** Implementar interfaz `IDescuentoStrategy` y el patrón `Strategy` para los tipos de cliente VIP, Frecuente, Regular y Nuevo, eliminando la cadena condicional y cumpliendo con el principio OCP.
4. **Commit 4 - [Arch]:** Definir abstracciones para los servicios de infraestructura (`IPedidoRepository`, `IFacturador`, `INotificador`) y migrar sus implementaciones para respetar los principios SRP y DIP.
5. **Commit 5 - [Integrate]:** Actualizar la clase principal `GestorPedidos` para inyectar sus dependencias vía constructor y corregir vulnerabilidad de inyección SQL mediante `PreparedStatement`.

8 Conclusión

La transición de un modelo transaccional procedural altamente acoplado a un diseño orientado a objetos basado en interfaces y patrones conductuales (*Strategy* y *Factory*) garantiza una estructura robusta. Esto permite a la organización tecnológica adaptarse a las demandas del mercado reduciendo el costo de cambio, eliminando errores colaterales comunes y mejorando significativamente la conformidad con la métrica ISO/IEC 25010 de mantenibilidad de software.